

Representación de datos y arquitectura de ordenadores

Pedro Vargas Pérez

Sistemas Informáticos 1º DAW-DAM

versión 1.1 Bajo licencia CC

Atribución-NoComercial-CompartirIgual 4.0 Internacional

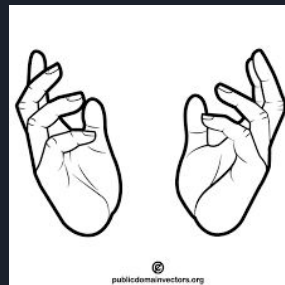


Índice

1. Representación de datos.
 - 1.1. [Sistemas de numeración](#)
 - 1.1.1. Bases 10, 2, 8 y 16 ([cambios desde base 10 y TFN](#))
 - 1.2. [Representación de datos en el ordenador \(binario\)](#)
 - 1.2.1. [Capacidad de representación en binario](#)
 - 1.2.2. [Patrones y múltiplos](#)
 - 1.2.3. [Múltiplos de bit y Bytes en el sistema internacional](#)
 - 1.3. Números
 - 1.3.1. [Naturales](#)
 - 1.3.2. [Enteros](#)
 - 1.3.3. [Reales](#)
 - 1.4. [Texto](#)
 - 1.5. [Imágenes](#)
 - 1.6. [Audio](#)
 - 1.7. [Vídeo](#)

Sistemas de numeración

Los seres humanos trabajamos en base 10 por la sencilla razón



El sistema de numeración **base 10** consta de 10 elementos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

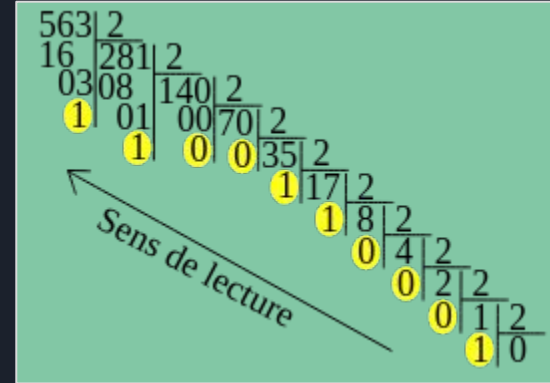
Los ordenadores en cambio trabajan en **base 2 o binario** cuyos únicos elementos son el 0 y el 1

Otros sistemas de numeración que resultan útiles en informática son:

- Sistema de numeración **octal o base 8**: 0, 1, 2, 3, 4, 5, 6, 7
- Sistema de numeración **hexadecimal o base 16**: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Cambios de base 10 a cualquier otra base

- 01 Dividimos el número en cuestión entre la base a la que lo queremos convertir (**división entera**)
- 02 Cogemos los restos en orden inverso al que han ido apareciendo
- 03 Anotamos como resultado ese valor (en el caso del ejemplo 563_{10} sería 1000110011_2)
- 04 Esto es aplicable a cualquier otra base de numeración a la que queramos convertir desde base 10





Teorema fundamental de la numeración T.F.N.

Este teorema establece la forma general de construir números en un sistema de numeración posicional. Primero estableceremos unas definiciones básicas:

- **N**: número válido en el sistema de numeración.
- **b**: base del sistema de numeración. Número de símbolos permitidos en el sistema.
- **d_i**: un símbolo cualquiera de los permitidos en el sistema de numeración.
- **n**: número de dígitos de la parte entera.
- **,**: coma fraccionaria. Símbolo utilizado para separar la parte entera de un número de su parte fraccionaria.
- **k**: número de dígitos de la parte decimal.

La fórmula general para construir un número N, con un número finito de decimales, en un sistema de numeración posicional de base b es la siguiente:

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^{n-1} d_i b^i \\ N = d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

Veamos un par de ejemplos de la aplicación del TFN

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^{n-1} d_i b^i \\ N = d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

Ejemplo en el sistema decimal

$$1492,36 = 1 \cdot 10^3 + 4 \cdot 10^2 + 9 \cdot 10^1 + 2 \cdot 10^0 + 3 \cdot 10^{-1} + 6 \cdot 10^{-2}$$

Ejemplo en el sistema binario

$$111_2 = 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 4 + 2 + 1 = 7_{10}$$



Conversión al peso

$$N = \begin{cases} \langle d_{(n-1)} \dots d_1 d_0, d_{-1} \dots d_{-k} \rangle = \sum_{i=-k}^{n-1} d_i b^i \\ N = d_{n-1} b^{n-1} + \dots + d_1 b^1 + d_0 b^0 + d_{-1} b^{-1} + \dots + d_{-k} b^{-k} \end{cases}$$

No deja de ser el uso del TFN pero ahorramos mucho tiempo con cosas que ya sabemos:

- La multiplicación por 0 siempre da 0: $n \cdot 0 = 0$
- La multiplicación por 1 da el elemento multiplicado: $n \cdot 1 = n$

Por tanto si queremos convertir 1011001_2 a base 10 podemos:

A. Usar TFN mecánicamente:

$$1011001_2 = 1 \cdot 2^6 + 0 \cdot 2^5 + 0 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 89$$

B. Usar TFN aplicando el peso

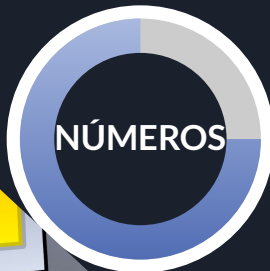
$$1011001_2 = 2^6 + 2^4 + 2^3 + 2^0 = 89$$





Representación de datos en sistemas informáticos

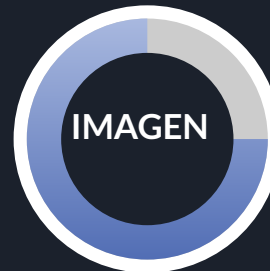
Los tipos de datos que digitalizamos y por tanto representamos en binario dentro de un sistema informático son:



NÚMEROS



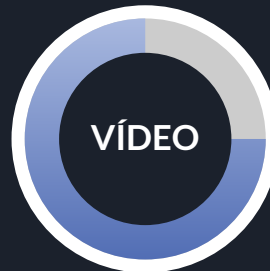
TEXTO



IMAGEN



AUDIO



VÍDEO





Capacidad de representación en base 2



bit

La mínima cantidad de memoria se denomina **bit** y se corresponde con una sola posición de memoria, cómo puede tomar dos valores diferentes (0 y 1) decimos que con un bit podemos representar dos informaciones diferentes.

Patrón de bits

Cualquier combinación de dos o más bits (n) con las que podré representar 2^n informaciones diferentes. (Ejemplo con 3 bits puedo representar $2^3 = 8$ combinaciones diferentes: 000, 001, 010, 011, 100, 101, 110, 111).

Nibble

Conjunto de 4 bits que nos permite representar 16 informaciones diferentes (correspondencia directa con base 16)

Byte

Conjunto de 8 bits también llamado octeto que nos permite representar 256 informaciones diferentes



Múltiplos (S.I. vs ISO/IEC 80000-13)

Tanto el **bit** como el **Byte se nos quedan pequeños enseguida**, de modo que tendremos que utilizar **múltiplos** de ambos (normalmente las **medidas en bits** se usan para hablar de **transmisión** de información y las **medidas en Bytes** para **almacenamiento**)

Hubo un tiempo en el que los informáticos hablábamos de **Kilobyte (o KB)** como el conjunto de **1024 Bytes**, lo cual creaba una discordancia con el sistema internacional que aplica al término **Kilo** como el conjunto de **1000** (ej. Kg 1000 gramos, Km 1000 metros)

En **2008** la organización internacional de estandarizaciones ISO, saca una norma específica para que los informáticos **no usemos el múltiplo kilo como 1024 (2^{10})**, de modo que cuando hablemos de kilo nos reframos a 1000 (10^3) y proponen **nuevos nombres** para los múltiplos basados en las potencias de 2 (Kibi, Mebi, ...)

Múltiplos de bits (ISO/IEC 80000-13)

b = bit B = byte; Kb= Kilobit, Kib = Kibibit.

Múltiplos de bits			
Sistema Internacional (decimal)		ISO/IEC 80000-13 (binario)	
Múltiplo (símbolo)	SI	Múltiplo (símbolo)	ISO/IEC
<u>kilobit (Kb)</u>	10^3	<u>kibibit (Kib)</u>	2^{10}
<u>megabit (Mb)</u>	10^6	<u>mebibit (Mib)</u>	2^{20}
<u>gigabit (Gb)</u>	10^9	<u>gibibit (Gib)</u>	2^{30}
<u>terabit (Tb)</u>	10^{12}	<u>tebibit (Tib)</u>	2^{40}
<u>petabit (Pb)</u>	10^{15}	<u>pebibit (Pib)</u>	2^{50}
<u>exabit (Eb)</u>	10^{18}	<u>exbibit (Eib)</u>	2^{60}
<u>zettabit (Zb)</u>	10^{21}	<u>zebibit (Zib)</u>	2^{70}
<u>yottabit (Yb)</u>	10^{24}	<u>yobibit (Yib)</u>	2^{80}

Múltiplos de Bytes (ISO/IEC 80000-13)

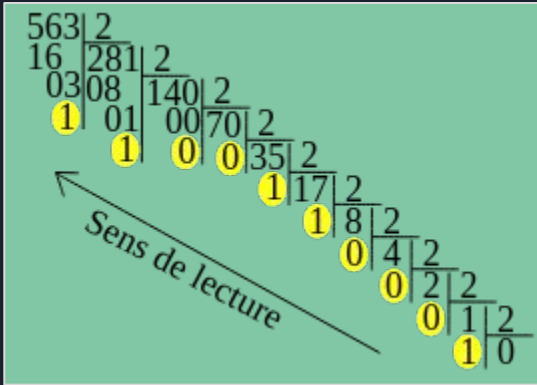
b = bit, B = byte; KB= Kilobyte, KiB = Kibibyte.

Múltiplos de bytes			
Sistema Internacional (decimal)		ISO/IEC 80000-13 (binario)	
Múltiplo (símbolo)	SI	Múltiplo (símbolo)	ISO/IEC
<u>kilobyte (KB)</u>	10^3	<u>kibibyte (KiB)</u>	2^{10}
<u>megabyte (MB)</u>	10^6	<u>mebibyte (MiB)</u>	2^{20}
<u>gigabyte (GB)</u>	10^9	<u>gibibyte (GiB)</u>	2^{30}
<u>terabyte (TB)</u>	10^{12}	<u>tebibyte (TiB)</u>	2^{40}
<u>petabyte (PB)</u>	10^{15}	<u>pebibyte (PiB)</u>	2^{50}
<u>exabyte (EB)</u>	10^{18}	<u>exbibyte (EiB)</u>	2^{60}
<u>zettabyte (ZB)</u>	10^{21}	<u>zebibyte (ZiB)</u>	2^{70}
<u>yottabyte (YB)</u>	10^{24}	<u>yobibyte (YiB)</u>	2^{80}

Representación de números naturales

Binario puro

Binario puro (no se usa como representación en el ordenador): se convierte la parte entera haciendo la división entera y cogiendo los restos en orden inverso, la parte decimal se va multiplicando la parte decimal y se van cogiendo los 0 o 1 que puedan surgir tras la multiplicación. **No permite** representación de números **negativos**.



$$\begin{aligned}0,640625 \cdot 2 &= 1,28125 \\0,28125 \cdot 2 &= 0,5625 \\0,5625 \cdot 2 &= 1,125 \\0,125 \cdot 2 &= 0,250 \\0,250 \cdot 2 &= 0,5 \\0,5 \cdot 2 &= 1\end{aligned}$$

Por tanto $563,640625_{10} = 1000110011,101001_2$



Representación de números naturales BCD (Binary-Coded Decimal)

Código BCD (Binary-Coded Decimal) (no es muy común su uso como representación actualmente). Se basa en que se asigna a cada dígito decimal 4 dígitos binarios. Se desperdicia espacio al encontrarnos con codificaciones que no se usan (1010, 1011, 1100, 1101, 1110, 1111)

Es difícil hacer operaciones en binario. **No permite** representación de números **negativos**.

Decimal	0	1	2	3	4	5	6	7	8	9
<u>BCD</u>	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001



Representación de números enteros Signo y magnitud

Esta es la primera notación que nos permite representar **números negativos**, para ello se divide los bits de los que dispongamos en dos bloques: **el bit de más a la izquierda** representará **el signo (0 positivo y 1 negativo)** el resto de los bits se usan para valor que se representa en binario puro. Nótese que disponemos de dos representaciones para el 0 (00000000 y 10000000 en el caso de un patrón de 8 bits) que sería +0 y -0.

Si disponemos de un patrón de **8 bits (1 Byte)** podemos representar **256 combinaciones diferentes**, como dos de ellas se usan para el 0 (+0 y -0), nos quedan 254 de las cuales serán 127 para números negativos y 127 para positivos. Puedo representar desde -127 hasta el +127 (con dos representaciones para el 0)

Si disponemos de un patrón de **10 bits** podemos representar **1024 combinaciones diferentes**, como dos de ellas se usan para el 0 (+0 y -0), nos quedan 1022 de las cuales serán 511 para números negativos y 511 para positivos. Puedo representar **desde -511 hasta el +511** (con dos representaciones para el 0)

Representación de números enteros **Signo y magnitud** Ejemplo con 2 Bytes

Valor decimal	Signo y magnitud		
	Signo	Magnitud	Representación en signo y magnitud 16 bits
-97	1	000000001100001	1000000001100001
59237	0	1110011101100101 No se puede representar faltan bits	No se puede representar faltan bits
97	0	000000001100001	0000000001100001
-326	1	000000101000110	1000000101000110

Tengo sólo 15 posiciones para la magnitud, porque una está reservada para el signo, al convertir 59237_{10} obtengo 1110011101100101_2 como no me queda espacio para el signo, no puede ser representado en signo y magnitud con 2 bytes



Representación de números enteros

Complemento a 1

Esta notación nos permite representar **números negativos**, en este caso los números **positivos** se dejan en **binario puro** (siempre y cuando el bit de más a la izquierda quede a 0) y los **negativos se complementan**, es decir los 0 se convierten en 1 y los 1 en 0. Sigue teniendo la misma dificultad que la anterior, tenemos dos representaciones para el 0 (con 8 bits 0000 0000 = +0 y 1111 1111 = -0)

Si disponemos de un patrón de **8 bits (1 Byte)** podemos representar **256 combinaciones diferentes**, como dos de ellas se usan para el 0 (+0 y -0), nos quedan 254 de las cuales serán 127 para números negativos y 127 para positivos. Puedo representar **desde -127 hasta el +127** (con dos representaciones para el 0)

Si disponemos de un patrón de **10 bits** podemos representar **1024 combinaciones diferentes**, como dos de ellas se usan para el 0 (+0 y -0), nos quedan 1022 de las cuales serán 511 para números negativos y 511 para positivos. Puedo representar **desde -511 hasta el +511** (con dos representaciones para el 0)

Representación de números enteros complemento a 1 Ejemplo con 1 Byte

Valor decimal	Representación en complemento a 1 8 bits
-97	10011110
-128	10000000 No se puede representar faltan bits
16	00010000
-16	1 1101111

Cuando convierto 128 a binario obtengo 1000 0000 si aplico el complemento sin darme cuenta de que no cabe tendría 0111 1111 que es el número +127. En complemento a 1 todo lo que empieza por 0 es positivo y lo que empieza por 1 es negativo.



Representación de números enteros

Complemento a 2

Esta notación nos permite representar **números negativos**, en este caso los números **positivos** se dejan en **binario puro** (siempre y cuando el bit de más a la izquierda quede a 0) y los **negativos se complementan con el complemento a 1 y se le suma 1**. En este caso tenemos ya **una sola representación para el 0**, y podremos aprovechar todo el espacio de representaciones.

Si disponemos de un patrón de **8 bits (1 Byte)** podemos representar **256 combinaciones diferentes**, tenemos ahora una sola representación para el 0, nos quedan 255 de las cuales serán 128 para números negativos y 127 para positivos. Puedo representar **desde -128 hasta el +127**.

Si disponemos de un patrón de **10 bits** podemos representar **1024 combinaciones diferentes**, tenemos ahora una sola representación para el 0, nos quedan 1023 de las cuales serán 512 para números negativos y 511 para positivos. Puedo representar **desde -512 hasta el +511**.

Representación de números enteros **complemento a 2** Ejemplo con 1 Byte

Valor decimal	Representación en complemento a 2 8 bits
-97	10011111
128	1 0000000 (8 bits) No se puede representar este sería un negativo, empieza por 1
16	00010000
-16	11110000

Cuando convierto 128 a binario obtengo 1000 0000 como empieza por 1 entonces claramente si lo almaceno sería un número negativo, concretamente -128 porque:

1. $128_{10} = 1000\ 0000_2$
2. El complemento de 1000 0000 es 0111 1111
3. Si a ese valor le sumo 1 obtengo 1000 0000 que es la representación de C-2 del -128



Representación de números enteros

Exceso 2^{n-1}

Esta notación nos permite representar **números negativos**, en este caso cualquiera que sea el número decimal, positivo o negativo le **sumaremos el exceso 2^{n-1}** donde **n es el número de bits del que disponemos** y pasaremos el valor resultante a binario puro. Hay que tener en cuenta dos cosas:

- El valor resultante de sumarle el exceso 2^{n-1} no puede ser negativo
- El valor resultante no puede superar el número n de bits disponibles.

Si disponemos de un patrón de **8 bits (1 Byte)** podemos representar desde el **-128 hasta el +127**, tenemos ahora una sola representación para el 0:

- **-128** + 2^{8-1} (128) = 0 → **0000 0000**
- **0** + 2^{8-1} (128) = 128 → **1000 0000**
- **-128** + 2^{8-1} (128) = 0 → **0000 0000**
- **+127** + 2^{8-1} (128) = 255 → **1111 1111**

Si disponemos de un patrón de **10 bits** podemos representar desde **-512 hasta el +511**.

Representación de números enteros **exceso 2^{n-1}**

Ejemplo con **1 Byte (exceso 128)**

Valor decimal	Representación en exceso 2^{n-1} 8 bits (exceso $2^7 = 128$)	
	Exceso 128	Representación
-97	$-97+128=31$	00011111
592	$592+128=720$	No se puede representar faltan bits
97	$97+128=225$	11100001
-16	$-16+128=112$	01110000

Cuando intento representar cualquier número superior a 127, al sumarle 128 obtengo un valor igual o superior a 256, que ya no es representable con 8 bits.

En el ejemplo $592 + 128 = 720$ que al pasarlo a binario nos da 10 1101 0000, el cual ocupa más de 8 bits.



Representación de números reales IEEE-754

El estándar IEEE 754 (conocido también como de **coma o punto flotante**) define 8 representaciones de punto flotante, de las cuales 5 de ellas son en binario, las más usadas e importantes son:

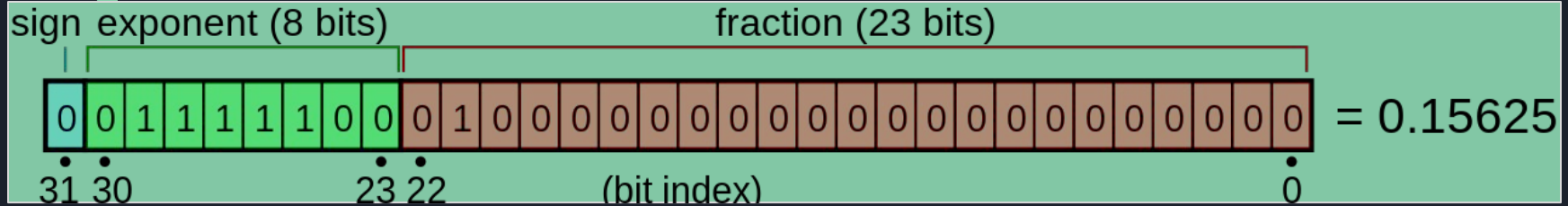
- Simple precisión (**binary32**)
- Doble precisión (**binary64**)

Aunque los dos formatos difieren en cuanto al número de bits que dedican a cada parte de la representación tienen elementos en común:

- El **bit más significativo** (el que queda más a la izquierda) es el **signo** del número.
- A continuación encontramos el exponente (8 u 11 bits dependiendo de la versión), el exponente se expresa en binario en **exceso $2^{n-1}-1$** (127 o 1023 dependiendo de la versión)
- A continuación encontramos la **mantisa o significando** (23 o 52 bits según la versión).

Representación de números reales IEEE-754

binary32



Veamos cómo pasamos el número decimal a binario y lo representamos en binary32:

1. **Convertimos** $0.15625_{10} \rightarrow 0.00101_2$
2. **Normalizamos**, desplazando la coma hasta que nos quede un 1 a su izquierda:
 $1.01 \cdot 2^{-3}$
3. Ya tenemos el **exponente** (-3) y el **significando** 01 (el uno de la parte entera no se coge y se rellena con ceros el resto de posiciones a la derecha)
4. Ahora ya podemos escribir el valor completo:
 - a. El **signo** 0 porque es positivo
 - b. El exponente $-3 + 2^{8-1}-1 (127) = 124_{10} \rightarrow 0111\ 1100_2$
 - c. El significando 010 0000 0000 0000 0000 0000

Diagram illustrating the IEEE 754 single-precision floating-point format. The 32-bit word is divided into three fields: sign (1 bit), exponent (8 bits), and fraction (23 bits). The bit positions 63, 52, and 0 are marked at the bottom.

Supongamos ahora que queremos almacenar $-0,2_{10}$ en binario en el formato binary64:

- [illegible]

Representación del texto en un S.I.

El primer problema que se nos plantea es cuantos bits dedicar para la representación de caracteres.

Los primeros que se plantean la necesidad de pasar los caracteres a binario son angloparlantes, y ellos tienen:

- 26 caracteres minúsculas.
- 26 caracteres mayúsculas.
- 10 caracteres numéricos
- Un puñado de símbolos:
?=(/(&%\$"!.,,:; ...

En total unos 85 caracteres diferentes, si lo intentara representar con 6 bits $2^6 = 64$, sería insuficiente, se necesitan como mínimo 7 bits $2^7 = 128$



Código ASCII (7 bits)

b₇ b₆ b₅ b₄ b₃ b₂ b₁ Bits					0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 1 0 1 1 1 0 1 1 1							
Column Row					0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	—	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL



ASCII extendido (8 bits)

Pronto comienza a verse que existen símbolos y caracteres de otros idiomas occidentales que no tienen cabida en esa codificación, se necesita un mayor espacio de representación, **añadiendo un solo bit podemos representar el doble de valores** diferentes:

Con 8 bits ya podemos representar $2^8 = 256$

Ahora ya sí entran otros símbolos que antes habían quedado excluidos como:

- Nuestra famosa ñ, las **vocales con tilde y la diéresis**.
- Algunos **símbolos gráficos**: ☒ | † ‡ ‖
- Se introducen también algunas **letras del alfabeto griego**: α β Γ π Σ
- **Símbolos matemáticos**: ± ≥ ≤

Tabla código ASCII extendido (8 bits)

The ASCII code

American Standard Code for Information Interchange

ASCII control characters			
DEC	HEX	Símbolo ASCII	
00	00h	NULL	(carácter nulo)
01	01h	SOH	(inicio encabezado)
02	02h	STX	(inicio texto)
03	03h	ETX	(fin de texto)
04	04h	EOT	(fin transmisión)
05	05h	ENQ	(enquiry)
06	06h	ACK	(acknowledgement)
07	07h	BEL	(timbre)
08	08h	BS	(retroceso)
09	09h	HT	(tab horizontal)
10	0Ah	LF	(salto de línea)
11	0Bh	VT	(tab vertical)
12	0Ch	FF	(form feed)
13	0Dh	CR	(retorno de carro)
14	0Eh	SO	(shift Out)
15	0Fh	SI	(shift In)
16	10h	DLE	(data link escape)
17	11h	DC1	(device control 1)
18	12h	DC2	(device control 2)
19	13h	DC3	(device control 3)
20	14h	DC4	(device control 4)
21	15h	NAK	(negative acknowledge.)
22	16h	SYN	(synchronous idle)
23	17h	ETB	(end of trans. block)
24	18h	CAN	(cancel)
25	19h	EM	(end of medium)
26	1Ah	SUB	(substitute)
27	1Bh	ESC	(escape)
28	1Ch	FS	(file separator)
29	1Dh	GS	(group separator)
30	1Eh	RS	(record separator)
31	1Fh	US	(unit separator)
127	20h	DEL	(delete)

ASCII printable characters											
DEC	HEX	Símbolo	DEC	HEX	Símbolo	DEC	HEX	Símbolo	DEC	HEX	Símbolo
32	20h	espacio	64	40h	@	96	60h	`	128	80h	Ç
33	21h	!	65	41h	A	97	61h	a	129	81h	ü
34	22h	"	66	42h	B	98	62h	b	130	82h	é
35	23h	#	67	43h	C	99	63h	c	131	83h	â
36	24h	\$	68	44h	D	100	64h	d	132	84h	ä
37	25h	%	69	45h	E	101	65h	e	133	85h	à
38	26h	&	70	46h	F	102	66h	f	134	86h	á
39	27h	'	71	47h	G	103	67h	g	135	87h	ç
40	28h	(	72	48h	H	104	68h	h	136	88h	ê
41	29h	)	73	49h	I	105	69h	i	137	89h	ë
42	2Ah	*	74	4Ah	J	106	6Ah	j	138	8Ah	è
43	2Bh	+	75	4Bh	K	107	6Bh	k	139	8Bh	ï
44	2Ch	,	76	4Ch	L	108	6Ch	l	140	8Ch	í
45	2Dh	-	77	4Dh	M	109	6Dh	m	141	8Dh	ì
46	2Eh	.	78	4Eh	N	110	6Eh	n	142	8Eh	Ä
47	2Fh	/	79	4Fh	O	111	6Fh	o	143	8Fh	Å
48	30h	0	80	50h	P	112	70h	p	144	90h	É
49	31h	1	81	51h	Q	113	71h	q	145	91h	æ
50	32h	2	82	52h	R	114	72h	r	146	92h	Æ
51	33h	3	83	53h	S	115	73h	s	147	93h	ó
52	34h	4	84	54h	T	116	74h	t	148	94h	ô
53	35h	5	85	55h	U	117	75h	u	149	95h	õ
54	36h	6	86	56h	V	118	76h	v	150	96h	ù
55	37h	7	87	57h	W	119	77h	w	151	97h	û
56	38h	8	88	58h	X	120	78h	x	152	98h	ý
57	39h	9	89	59h	Y	121	79h	y	153	99h	Ö
58	3Ah	:	90	5Ah	Z	122	7Ah	z	154	9Ah	Ü
59	3Bh	;	91	5Bh	[	123	7Bh	{	155	9Bh	ø
60	3Ch	<	92	5Ch	\	124	7Ch	}	156	9Ch	£
61	3Dh	=	93	5Dh	]	125	7Dh	}	157	9Dh	Ø
62	3Eh	>	94	5Eh	^	126	7Eh	~	158	9Eh	x
63	3Fh	?	95	5Fh	-				159	9Fh	f

theASCIICode.com.ar

Extended ASCII characters											
DEC	HEX	Símbolo	DEC	HEX	Símbolo	DEC	HEX	Símbolo	DEC	HEX	Símbolo
160	A0h	á	192	C0h	Ł	224	E0h	Ó			
161	A1h	â	193	C1h	ł	225	E1h	ô			
162	A2h	ã	194	C2h	Ł	226	E2h	ö			
163	A3h	ä	195	C3h	ł	227	E3h	õ			
164	A4h	å	196	C4h	Ł	228	E4h	ø			
165	A5h	Ä	197	C5h	ł	229	E5h	Ö			
166	A6h	Å	198	C6h	Ł	230	E6h	µ			
167	A7h	Æ	199	C7h	ł	231	E7h	þ			
168	A8h	Ç	200	C8h	Ł	232	E8h	ÿ			
169	A9h	ç	201	C9h	ł	233	E9h	ÿ			
170	AAh	Ð	202	CAh	Ł	234	EAh	ÿ			
171	ABh	ð	203	CBh	ł	235	EBh	ÿ			
172	ACH	Ñ	204	CAh	Ł	236	ECh	ÿ			
173	ADh	ñ	205	CDh	ł	237	EDh	ÿ			
174	AEh	Ò	206	CEh	Ł	238	EEh	ÿ			
175	AFh	ó	207	CFh	ł	239	EFh	ÿ			
176	B0h	Ô	208	D0h	Ł	240	F0h	ÿ			
177	B1h	õ	209	D1h	ł	241	F1h	±			
178	B2h	Ö	210	D2h	Ł	242	F2h	²			
179	B3h	ö	211	D3h	ł	243	F3h	³			
180	B4h	Ù	212	D4h	Ł	244	F4h	¼			
181	B5h	ú	213	D5h	ł	245	F5h	½			
182	B6h	û	214	D6h	Ł	246	F6h	¾			
183	B7h	ü	215	D7h	ł	247	F7h	¸			
184	B8h	Ý	216	D8h	Ł	248	F8h	¸			
185	B9h	ÿ	217	D9h	ł	249	F9h	¸			
186	BAh	À	218	DAh	Ł	250	FAh	¸			
187	BCh	Á	219	DBh	ł	251	FBh	¸			
188	BCh	Â	220	DC	Ł	252	FBh	¸			
189	BDh	Ã	221	DDh	ł	253	FDh	¸			
190	BEh	Ä	222	DEh	Ł	254	FEh	¸			
191	BFh	Å	223	DFh	ł	255	FFh	¸			



Antes de UNICODE

Cuando ya parece todo solucionado, enseguida comienza a aparecer el problema de otros alfabetos distintos al occidental.

En un primer momento Microsoft e ISO intentan solucionarlo con distintos conjuntos de caracteres que con 8 bits tienen todos los caracteres de un determinado alfabeto.

Problemas:

- No se pueden mezclar símbolos de distintos alfabetos
- Hay que establecer la página de códigos adecuada antes de codificar o decodificar la información.



Codificaciones ISO

- ISO 8859:
 - ISO 8859-1 Europa occidental
 - ISO 8859-2 Europa occidental y Centroeuropa ([checo](#), polaco, croata, rumano, esloveno, ...)
 - ISO 8859-3 Europa occidental y Europa del Sur
 - ISO 8859-4 Europa occidental y países bálticos (lituano, estonio y [lapón](#))
 - ISO 8859-5 alfabeto cirílico
 - ISO 8859-6 árabe
 - ISO 8859-7 griego
 - ISO 8859-8 Hebreo
 - ISO 8859-9 Europa occidental con el juego de caracteres turco
 - ISO 8859-10 Europa occidental con juegos de caracteres nórdicos, incluyendo el de [Islandia](#).
 - ISO 8859-11 tailandés
 - ISO 8859-13 idiomas bálticos y polaco
 - ISO 8859-14 idiomas celtas (gaélico irlandés, escocés, galés)
 - ISO 8859-15 Añade el símbolo de Euro y otros a ISO 8859-1
 - ISO 8859-16 idiomas centroeuropeos (polaco, checo, esloveno, eslovaco, húngaro, albano, ruma



UNICODE

Unicode es un proyecto que se inicia en 1987 por un par de ingenieros, uno de Apple y otro de Xerox.

El término Unicode proviene de los tres objetivos perseguidos:

- Universalidad: sirva para todos los alfabetos (habidos y por haber)
- Uniformidad: en el modo de definir cada carácter
- Unicidad: una única representación.

Desde un punto de vista técnico Unicode es compatible con codificaciones previas como ASCII7 o ISO8859-1

Número de bits para UNICODE:

- En un principio se prevén 16 bits
- Finalmente se deciden por codificaciones de hasta 32 bis. $2^{32} = 4.294.967.296$ posibilidades diferentes.



UNICODE: diferentes codificaciones

UTF-8: utiliza símbolos de **longitud variable de 1 a 4 bytes** por carácter. Puede representar cualquier carácter UNICODE, cualquier elemento ASCII 7 bits se representa igual (pero con 8 bits). Especialmente utilizado para transmisión, ya sean correos electrónicos, páginas web,...

UTF-16: utiliza símbolos de **longitud variable de 1 o 2 palabras de 16 bits** por carácter. Puede representar cualquier carácter UNICODE. Contiene la **gran mayoría de caracteres y sistemas de escritura en uso en la actualidad** con solo 16 bits.

UTF-32: utiliza símbolos de **longitud fija, siempre 32 bits** por carácter. La ventaja de tener una longitud fija es que los caracteres están indexados y que su búsqueda en la tabla es muy rápida.



UNICODE

What is Unicode?

ما هي الشفرة الموحدة "يونيكود" ؟

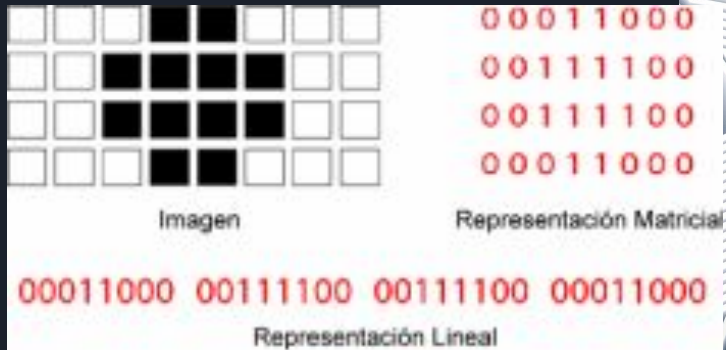
Was ist Unicode?

Что такое Unicode?

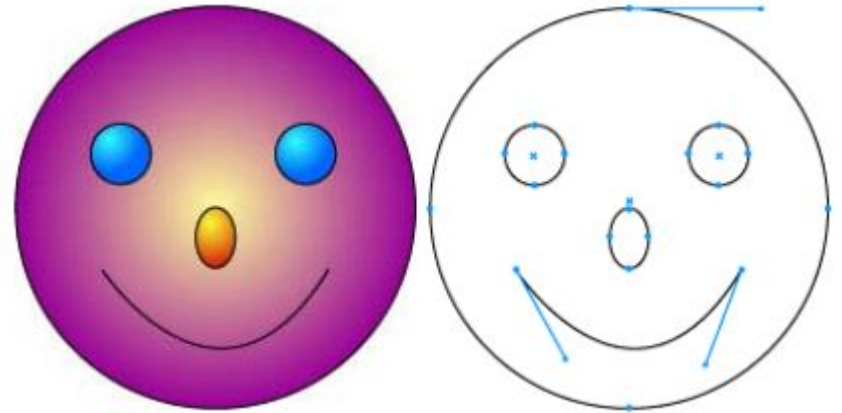
什么是Unicode?

Imágenes

Mapa de bits



Gráficos vectoriales





Imágenes vectoriales

La principal **ventaja** de las imágenes vectoriales es que **no tiene dependencia con el número de pixels** que dediquemos a dicha imagen.

La imagen se crea mediante **referencias matemáticas**. Es decir, en vez de trazar una retícula con miles o millones de puntos para trazar una línea, le decimos a la máquina unas coordenadas x_1 e y_1 le decimos que trace una línea hasta otras coordenadas x_2 e y_2 .

Así podemos dibujar círculos, cuadrados, triángulos y miles de formas. Sin entrar en detalles, diremos que esa es la base de los llamados dibujos vectoriales.

El principal **inconveniente** es que **estamos limitados en cuanto a lo que podemos dibujar o representar** con este tipo de imágenes.



Imágenes de mapas de bits

Vamos a representar cada píxel con uno o más bits, dependiendo del tipo de imagen que queramos almacenar. Veamos algunas definiciones previas:

Píxel: es la **menor unidad homogénea** (del mismo color) que forma parte de una imagen digital.

Imagen binaria: es una imagen digital que tiene únicamente dos valores posibles para cada píxel. **Normalmente**, los colores utilizados para su representación son **negro y blanco**, aunque puede usarse cualquier pareja de colores. Uno de los colores se emplea como fondo y el otro para los objetos que aparecen en la imagen.

Escala de grises: es una escala empleada en la imagen digital en la que el valor de cada píxel posee un valor equivalente a una graduación de gris, desde el blanco al negro, habitualmente se establecen 256 tonos diferentes (1 byte)

Color RGB: **Red, Green, Blue**, se establece para cada píxel la combinación de 256 tonos de cada uno de los colores ($8 \times 3 = 24 \text{ bits}$ o lo que es lo mismo **3 Bytes**)

Color CMYK: cyan, magenta, amarillo y negro ($8 \times 4 = 32 \text{ bits}$, **4 Bytes**)

Imagen en blanco y negro

Cada pixel debe tomar uno de dos valores, 1 o 0 que representarán el blanco y el negro.

Es posible tener una imagen binaria en cualesquiera otros dos colores

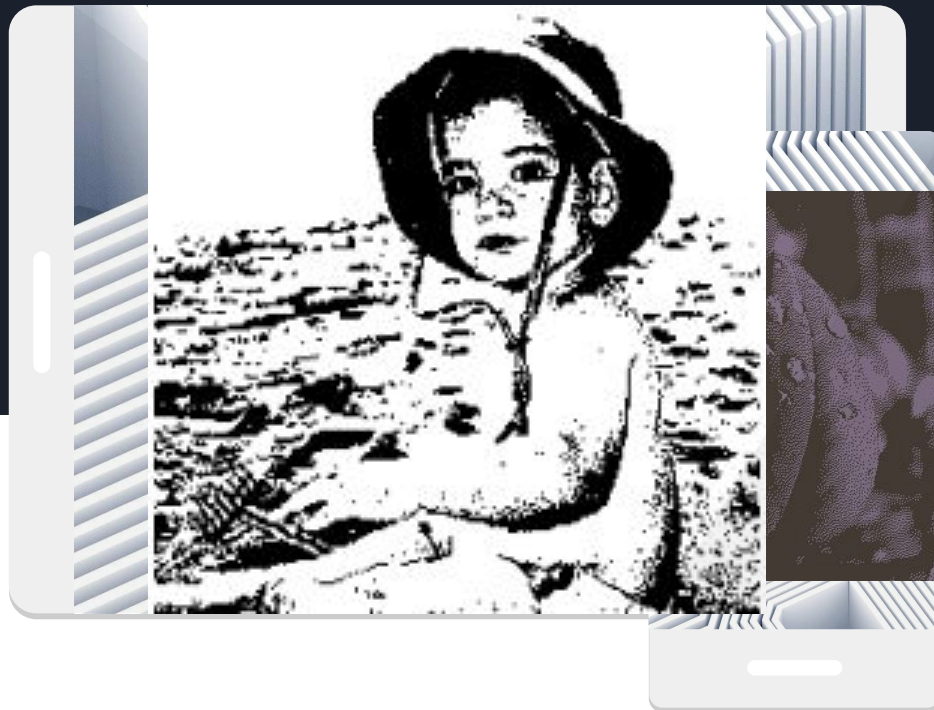


Imagen en escala de grises



Aunque en principio cualquier escala es posible se suele usar 8 bits, 1 Byte por cada pixel.

Son un total de 254 tonos + el blanco y el negro 256 tonos diferentes desde el blanco (1111 1111) hasta el negro (0000 0000).

Imagen en color RGB

Cada pixel debe tomar uno de las 2^{24} posibles combinaciones de color.

	R	V	A
Rojo (con 100% intensidad) →	11111111	00000000	00000000
Verde (con 100% intensidad) →	00000000	11111111	00000000
Azul (con 100% intensidad) →	00000000	00000000	11111111
Blanco (con 100% intensidad) →	11111111	11111111	11111111





Cuánto ocupan las imágenes

Si tuviésemos que almacenar toda la información de cada pixel sin comprimir agotaríamos muy rápidamente los recursos de nuestros dispositivos digitales.

Pongamos por **ejemplo**:

- Una **cámara actual con 48 Mpx** (48 millones de pixels) en una sola imagen o fotograma.
- RGB **3 Bytes por cada pixel**
- Una fotografía ocuparía **144 MB**

Afortunadamente tenemos distintos formatos de compresión de imágenes:

- **Sin pérdida**: la imagen se puede recuperar en toda su calidad. (**PNG**)
- **Con pérdida**: se pierden detalles pero se consiguen tamaños de archivos muy pequeños (**JPEG**).
- **WEBP**: es una alternativa relativamente nueva para imágenes en la web desarrollada por Google. Este formato utiliza una combinación de compresión sin pérdida y con pérdida para lograr tamaños de archivo más pequeños que los formatos de imagen anteriores.



Digitalización de audio

El audio que escuchamos o producimos es siempre analógico, sin embargo hoy en día lo almacenamos casi siempre en formato digital, para ello hacemos un proceso llamado digitalización en el que conseguiremos mayor o menor calidad dependiendo de:

- **Frecuencia de muestreo:** es el **conjunto de muestras** de la fuente de audio y de las ondas sonoras a intervalos periódicos. Clásicamente las frecuencias de muestreo utilizadas en los medios digitales han sido **44,1 kHz y 48 kHz**, ya que son capaces de cubrir todo el espectro de frecuencias audibles por el oído.
- **Cuantificación:** se mide el **voltaje que corresponde a la amplitud de cada muestra** y se le asigna un valor numérico discreto por aproximación (cuya precisión dependerá del bitrate de codificación).
- **Codificación:** convertimos esas cantidades a binario.
- **Profundidad de bits:** cuanta más profundidad de bits, mayor número de divisiones verticales en los que se puede tomar la muestra y por tanto mayor calidad.

Digitalización de audio



videoclases de

REPRESENTACIÓN DIGITAL
de la INFORMACIÓN

ALBERTO PRIETO ESPINOSA, ATC UGR

Módulo RD3. Audio Digital

4 Digitalización de una señal de audio



Codecs de audio

Códec: es un programa o dispositivo hardware capaz de **codificar o decodificar** una señal o flujo de datos digitales. Su uso está muy extendido para la codificación de señales de audio y video dentro de un formato contenedor.

Códecs de audio **con pérdidas:** En audio se aprovechan las **limitaciones del sistema auditivo humano** para comprimir los datos de audio. El nivel de compresión se puede controlar y depende de la calidad que se quiera obtener, el tamaño del fichero, el ancho de banda de la red, el tiempo de compresión... Usualmente se utilizan **compresiones máximas** para transmisiones, especialmente cuando son servicios en directo como **telefonía** o reproducciones de **audio desde la web**.

Códecs de audio **sin pérdidas:** Estos códecs utilizan una compresión sin pérdidas para minimizar el tamaño del flujo de datos. Para realizar esta codificación se utilizan **algoritmos basados en la eliminación de la redundancia** de la señal de audio, y por lo tanto en el grado de predictibilidad de la información. Si la señal tiene patrones repetitivos, este es redundante y por lo tanto fácil de predecir.



Características de los principales códecs de audio

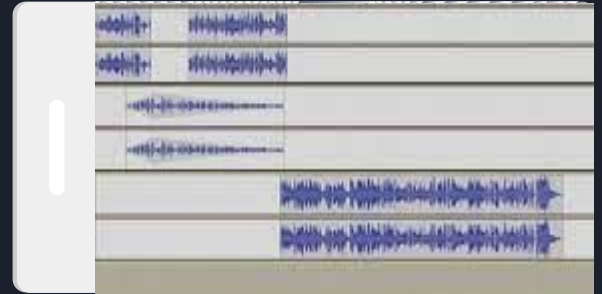
Codec	AAC	FLAC	Monkey's Audio	MP3	Vorbis (Ogg)	WavPack	WMA
Creador	ISO/IEC MPEG Audio Committe	Xiph.Org Foundation, Josh Coalson	Matthew T. Ashland	ISO/IEC MPEG Audio Committe	(Xiph.Org Foundation)	Conifer Software	Microsoft
Última version estable	ISO/IEC 14496-3	1.3.2	4.33	ISO/IEC 11172-3,ISO/IEC 13818-3	1.3.6	5.1.0	11.0
Tipo de compresión	Con pérdidas, Híbrido	Sin pérdidas	Sin pérdidas	Con pérdidas	Con pérdidas	Con pérdidas, Sin pérdidas, Híbrido	Con pérdidas, Sin pérdidas

Contenedores multimedia

Formato contenedor: es un tipo de formato de archivo que almacena información de vídeo, audio, metadatos e información de sincronización y corrección de errores siguiendo un formato preestablecido en su especificación técnica.

Las pistas de vídeo y audio suelen ir **comprimidas**, siendo distintos **códecs**, utilizados dentro de cada uno de los contenedores, los encargados de descomprimir la información en aras a su reproducción.

Cuando se crea un contenedor, en primer lugar se produce la **codificación de las pistas** y posteriormente son "**unidas**" (**multiplexadas**) siguiendo un patrón típico de cada formato.





Digitalización de vídeo

Un caso particular de imágenes son los vídeos que **normalmente incluyen una sucesión de imágenes junto con audio**.

El video es una representación de imágenes (llamadas **cuadros o frames**) en el tiempo. Una película es una serie de cuadros desplegados uno tras otro para crear la ilusión de movimiento. Así que si se sabe cómo almacenar una imagen dentro de un ordenador, también se sabe cómo almacenar un video; cada imagen o cuadro cambia a una serie de patrones de bits y se almacena. La combinación de las imágenes representa el video.

Del mismo modo que las imágenes o el audio **el vídeo también se comprime (codifica)** para conseguir tamaños más reducidos de archivos tanto para su almacenamiento como para su transferencia.



Principales códecs de vídeo

El problema que se pretende acometer con los códecs es que **la información de video es bastante ingente en relación con lo que un ordenador normal es capaz de manejar**. Es así como un par de segundos de video en una resolución apenas aceptable puede ocupar un lugar respetable en un medio de almacenamiento típico (disco duro, CD, DVD) y su manejo (copia, edición, visualización) puede llevar fácilmente a sobrepasar las posibilidades de dicho ordenador o llevarlo a su límite.

Entre los diversos tipos de codecs podemos reseñar: **Xvid, Theora, RealVideo, WMV, MPEG-1, MPEG-2, MPEG-4, H264, ON2...**

Formatos contenedores

Son un tipo de **formato de archivo** que almacena información de **vídeo, audio, metadatos e información de sincronización y corrección de errores** siguiendo un formato preestablecido en su especificación técnica.



Diagram illustrating the relationship between container formats and audio codecs. The container formats are categorized into three groups: Lossy compression, No compression, and Lossless compression. The audio codecs are categorized into three groups: Lossy compression, No compression, and Lossless compression. The diagram shows that container formats can support various audio codecs, and the support is indicated by 'Yes', 'No', or 'Not officially'.

	Lossy compression							No compression		Lossless compression			
	MP3	WMA	Vorbis	Opus	AAC	AC-3	DTS	PCM	LPCM	FLAC	ALAC	MLP / Dolby TrueHD	DTS-HD
AVI	Yes	Yes ^[36]	Not officially	?	Yes ^[37]	Yes	Yes	Yes	Yes	No	No	No	?
Matroska	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
MP4	Yes	Yes	Not officially	Yes	Yes	Yes ^[38]	Yes	No	Yes	Not officially ^[39]	Yes	No	Yes ^[40]
MPG	Yes	?	?	No	?	?	?	?	Yes	?	?	?	?
MXF	Yes	No	?	?	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Yes
Ogg	Yes	No	Yes	Yes	No	No	No	Yes	Yes	Yes	No	No	No
QuickTime	Yes	Yes	Yes	?	Yes		?	Yes	Yes	Yes	Yes	No	No

Formatos contenedores de vídeo

Contenedores

Códecs de vídeo

	MPEG-1	MPEG-2	MPEG-4 (A)SP	H.264/ MPEG-4 AVC	H.265/ HEVC	VC-1/ WMV	Real Video	Theora	Microsoft MPEG4 V2	VP8	VP9	MVC
AVI	Yes	Yes	Yes	Yes ^[41]	No	Yes	Yes	Yes	Yes	Yes	No	?
Matroska	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes ^[42]
MP4	Yes	Yes	Yes	Yes	Yes	Yes ^[43]	Yes	Yes	Yes	Yes	Yes	Yes
MXF	Yes	Yes	Yes	Yes	?	Yes	No	?	?	?	?	?
Ogg	Yes	Yes	Yes	Yes	?	Yes	No	Yes	?	?	?	?
QuickTime	Yes	Yes	Yes	Yes	Yes ^[44]	Yes	No	Yes	?	?	?	?